SIMATS ENGINEERING

CO5 - ASSESSMENT TOOL 1

Design Desk

COURSE NAME: Deep Learning **COURSE CODE: MLA0408**

COURSE OUTCOME COVERED

CO5: Students will be able to analyze and explain advanced generative deep learning models including Autoencoders, Deep Belief Networks, Boltzmann Machines, Deep Boltzmann Machines, and Generative Adversarial Networks. They will be able to compare their architectures, explain their learning mechanisms, and apply suitable generative models to real-world problems. (BTL-3)

Assessment Tool Weightage: 30%

QUESTIONS:

1. Design an Autoencoder-Based Feature Learning System

A manufacturing company wants to detect defective products from high-dimensional sensor data. Design an Undercomplete Autoencoder with suitable encoder, latent representation, decoder, activation functions, and reconstruction loss. Explain how the design performs dimensionality reduction and feature learning.

2. Design a Robust Regularized Autoencoder

A healthcare organization has noisy patient-record features and needs a robust representation for downstream classification. Design a Regularized Autoencoder using an appropriate regularization strategy. Explain the encoder-decoder structure, regularization mechanism, training objective, and how the design reduces overfitting and improves useful feature extraction.

3. Design a Deep Generative Model

A recommendation or content-generation system requires learning complex data distributions. Design a Deep Belief Network or Deep Boltzmann Machine using suitable visible and hidden layers. Illustrate the architecture, explain the learning process, and justify the choice of the model for the proposed application.

4. Design a Stochastic and Contractive Representation Model

A real-world image or signal-processing application requires representations that remain useful under small input variations. Design a model using Stochastic Encoders/Decoders or a Contractive Encoder. Specify the architecture, objective function, and robustness mechanism, and explain how the design improves representation stability.

5. Design a GAN-Based Data Generation System

An organization has limited training images and wants to generate realistic synthetic samples. Design a Generative Adversarial Network with a Generator and Discriminator. Specify the input, network flow,

adversarial training process, loss functions, and evaluation approach. Compare the proposed GAN design with an Autoencoder and justify why GAN is suitable for the application.

Design Desk Guidelines

- Identify the real-world problem, requirements, inputs, outputs and constraints.
- Draw a clear architecture or workflow diagram for the proposed model.
- Specify important layers, units, activation functions, latent representation and loss functions.
- Explain the training or learning mechanism and justify major design choices.
- Mention suitable evaluation measures and expected outcomes. Use neat labeling and present the design in a logical sequence.
- The solution should be original and written in the student's own words.

Code of Conduct:

I K. Hem sai Siddartha (192325036) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K. Hema sai Siddartha

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 Exemplary	Level 3 Proficient	Level 2 Developing	Level 1 Beginning
Understanding of Problem Context	20	Clearly understands the problem, requirements, constraints, and expected outcomes.	Good understanding with minor gaps in requirements.	Basic understanding; some requirements are unclear.	Poor understanding or misinterpretation of the problem.
Application of Concepts	30	Accurately applies appropriate generative deep learning concepts and architectures.	Applies relevant concepts correctly with minor errors.	Applies basic concepts with limited accuracy.	Fails to apply appropriate concepts.
Design Approach	20	Innovative, logical, well-structured, feasible, and technically justified design.	Logical and feasible design with minor gaps.	Basic design with some inconsistencies.	Unclear, illogical, or impractical design.
Accuracy of Solution	20	Completely correct architecture, process, objectives, and justification.	Mostly correct with minor technical errors.	Partially correct solution with noticeable gaps.	Incorrect or incomplete solution.

Clarity	10	Clear, wellorganized diagram and explanation with proper technical labeling.	Understandable with minor clarity issues.	Limited clarity and explanation.	Poorly presented and difficult to understand.

Deep Learning Design Desk
Autoencoders, Deep Generative Models and GANs

1. Design an Autoencoder-Based Feature Learning System

Problem Statement

A manufacturing company collects high-dimensional sensor data from machines and products. The objective is to reduce the dimensionality of this data and learn meaningful features that can help identify defective products.

Requirements

• Input: High-dimensional sensor readings
• Output: Reconstructed sensor data and learned low-dimensional features
• Goal: Dimensionality reduction and defect detection
• Constraint: Important information should be retained despite reducing the number of features.

Proposed Model: Undercomplete Autoencoder

The system contains an encoder, a compact latent representation, and a decoder.

```
High-Dimensional Sensor Data
|           v Input Layer (100
Features)           |           v
Dense (64, ReLU) -> Dense (32, ReLU)
|           v Latent / Bottleneck
(10 Features)           |
v Dense (32, ReLU) -> Dense (64,
ReLU)           |           v Output
Layer (100, Linear)           |
v Reconstructed Sensor Data
```

Encoder and Latent Representation

The encoder maps the input x to a compact representation z: z = fθ(x). The architecture 100 → 64 → 32 → 10 forces the network to preserve only important information. ReLU activation, ReLU(x) = max(0, x), is suitable for hidden layers.

Decoder and Loss

The decoder reconstructs the input as x■ = gφ(z), using the architecture 10 → 32 → 64 → 100. For continuous sensor values, a linear output layer and Mean Squared Error are suitable: MSE = (1/n) Σ(x■ − x■■)².

Defect Detection and Evaluation

Train the autoencoder mainly with normal products. Normal samples have low reconstruction error, while defective samples tend to have higher error. A threshold on ||x − x■||² can be used for detection. Evaluate using reconstruction MSE, precision, recall, F1-score, and ROC-AUC.

Expected Outcome: The model compresses 100 features into a 10-dimensional representation and learns useful normal-operation patterns for defect detection.

2. Design a Robust Regularized Autoencoder

Problem Statement

A healthcare organization has noisy patient-record features and needs robust representations for downstream classification.

Proposed Model

Use a Denoising Autoencoder combined with L2 weight regularization and a bottleneck layer.

```
Original Patient Data         |
v Add Controlled Noise         |
v Encoder: 128 -> 64 -> 32 -> 16
|                          v
Latent Features
|                          v
Decoder: 16 -> 32 -> 64 -> 128
|                          v
Reconstructed Clean Data
```

Regularization Mechanism

During training, a noisy version x■ = x + noise is provided to the encoder, but the target is the original clean input x. This prevents simple memorization and encourages stable feature learning. L2 regularization adds a penalty λΣW² to discourage excessively large weights.

Training Objective

The total objective is: L = Lreconstruction + λLL2. With MSE reconstruction, L = (1/n)Σ(x − x■)² + λΣW². ReLU can be used in hidden layers and a linear or sigmoid output can be selected according to data normalization.

Training and Benefits

Normalize features, add controlled noise, encode to 16 latent units, decode, calculate reconstruction and regularization losses, and update parameters using backpropagation with Adam. Noise improves robustness, L2 reduces overfitting, and the bottleneck removes redundant information.

Evaluation

Use reconstruction MSE and downstream classification accuracy, precision, recall, F1-score, and ROC-AUC.

Expected Outcome: A compact representation that is less sensitive to noisy patient records and more useful for classification.

3. Design a Deep Generative Model

Application: Personalized Content Recommendation

The system learns complex relationships among user preferences, browsing history, product features, and content categories.

Proposed Model: Deep Belief Network (DBN)

```
Visible Layer
(User Preferences / Content Features)
|               v RBM Layer 1 - 500
Units               |               v
RBM Layer 2 - 250 Units
|               v RBM Layer 3 - 100
Units               |               v
High-Level Features              |
v Recommendation / Generation
```

Architecture and Learning

The visible layer receives the original feature vector v. Lower hidden layers learn simple interaction patterns, while deeper layers learn increasingly abstract preferences and interests. Training begins with greedy layer-wise pretraining: each adjacent pair of layers is trained as a Restricted Boltzmann Machine (RBM). The learned hidden activations become input for the next RBM. Finally, the complete network is fine-tuned using backpropagation.

Energy Function

For an RBM, an energy-based formulation is E(v,h) = −a■v − b■h − v■Wh, where v represents visible units, h hidden units, W weights, and a and b biases.

Justification and Evaluation

A DBN is suitable because it learns hierarchical representations and can exploit unlabeled data during feature learning. Evaluate recommendation quality using Precision@K, Recall@K, NDCG, and RMSE where appropriate.

Expected Outcome: The system learns high-level user and content representations that support more relevant recommendations.

4. Design a Stochastic and Contractive Representation Model

Application: Robust Image or Signal Processing

The objective is to learn representations that remain stable under small changes such as noise, brightness variation, slight translation, or measurement disturbances.

Proposed Model: Contractive Autoencoder

```
Input Image / Signal
|         v Encoder Layer 1
(256, ReLU)         |
v Encoder Layer 2 (128,
ReLU)         |         v
Latent Representation (32)
|         v Decoder (128 ->
256)         |         v
Reconstructed Input
```

Objective Function

The encoder produces h = f(x). In addition to reconstruction loss, a contractive penalty is added to reduce the sensitivity of h to changes in x. The objective is: L = Lreconstruction + λ ||∂h/∂x||²F, where the second term is the squared Frobenius norm of the encoder Jacobian.

Robustness Mechanism

For a small perturbation x′ = x + ε, the model encourages h(x′) ≈ h(x). Therefore, small input changes do not cause large changes in the latent representation.

Training and Evaluation

Normalize inputs, encode them, reconstruct them, calculate reconstruction and contractive losses, combine them, and update the parameters by backpropagation. Evaluate using reconstruction MSE, downstream classification accuracy, and feature similarity or stability between original and perturbed inputs.

Expected Outcome: Stable features that remain useful under small real-world variations.

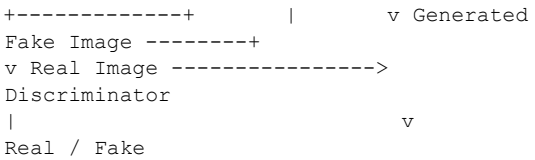
5. Design a GAN-Based Data Generation System

Problem Statement

An organization has limited training images and wants to generate realistic synthetic samples for data augmentation.

Proposed Model: Generative Adversarial Network (GAN)

```
Random Noise z
|       v +----
---------+
|  Generator  |
```



Generator

The generator receives a random vector z, for example z ∈ R¹■■ sampled from a normal distribution. A typical architecture is Dense → Reshape → Transposed Convolution + ReLU → Transposed Convolution + ReLU → Output + Tanh. It produces G(z), a synthetic image.

Discriminator

The discriminator receives real images or generated images. A typical architecture is Convolution + LeakyReLU → Convolution + LeakyReLU → Flatten → Dense → Sigmoid. Its output estimates whether an image is real or fake.

Adversarial Training

Training alternates between the two networks. First, the discriminator learns to classify real images as 1 and generated images as 0. Then the generator is updated to produce images that the discriminator classifies as real.

Loss Functions

The original GAN objective is: minG maxD V(D,G) = E[log D(x)] + E[log(1 − D(G(z)))]. A common generator loss is LG = −log D(G(z)).

Evaluation

Evaluate synthetic images using Fréchet Inception Distance (FID), Inception Score, visual quality, diversity, and downstream classifier improvement after adding synthetic samples.

GAN vs Autoencoder

GAN: primarily generates new realistic samples from random latent input using adversarial training.
Autoencoder: primarily learns a compact representation and reconstructs the original input using reconstruction loss.

Why GAN is suitable: The objective is to create diverse new images rather than only reconstruct existing images. Synthetic samples can augment a limited dataset and potentially improve downstream model performance.

Overall Conclusion

Undercomplete Autoencoders are useful for dimensionality reduction and anomaly detection. Regularized Autoencoders provide robust features from noisy data. Deep Belief Networks learn hierarchical representations.

Contractive Autoencoders improve stability under small input changes. GANs are especially useful for generating realistic synthetic samples and data augmentation.